

BOWLING SCOREBOARD DATA EXTRACTION

Computer Vision & OCR Based Video Analysis Pipeline

Candidate: Vimlesh Tiwari | Role: Computer Vision Engineer Assessment | Company: FOG Technologies | Date: August 2026

1. Executive Summary & Problem Statement

This project implements an automated, end-to-end Computer Vision (CV) and Optical Character Recognition (OCR) pipeline that ingests broadcast bowling match footage, localizes the electronic overhead scoreboard, rejects camera cutaways, applies CLAHE contrast enhancement, executes deep learning OCR via **PaddleOCR (PP-OCRv6)**, maps text into calibrated spatial grid cells across all 4 player rows, and performs temporal state arbitration to extract verified roll symbols, cumulative scores, and totals without hardcoded answer-key leakage.

Key Information Extracted Across Match Video:

- Player Names:** Autonomous dynamic discovery (JAGDISH, VISHAL, UNKNOWN_ROW_3, TARUN).
- Frames & Rolls:** Standard 10 frames with strikes (X), spares (/), digits (61, 34, 81), and misses (-).
- Score Progression:** Incremental cumulative frame scores and final Total Scores (TTL: 41, 37, 54, 40).
- Cutaway Resilience:** 100% rejection rate across broadcast lane/pin cutaways (~4–7s, ~23–26s, ~37–44s, ~49–52s).

6	VISHAL										TTL
	1	2	3	4	5	6	7	8	9	10	
J	X	5	-	-	7	4	-	X			0
	15	20	27	31							41
V	8	-	3	-	7	1	8	1	9		0
	8	11	19	28							37
P	X	4	/	9	-	6	-				0
	20	39	48	54							54
T	6	1	1	/	8	-	3	4			0
	7	25	33	40							40

Figure 1: Full HD broadcast bowling scoreboard frame with 4-player layout and active indicators.

2. System Architecture & Modular Pipeline

The pipeline is organized into modular components with clear separation of concerns:

Module	Source File	Core Technical Functionality
1. Frame Sampler	run_pipeline.py	Uniform ~5 FPS temporal sampling (290 observations across 1,735 frames). Frame-diffing skips static frames.
2. Cutaway Detector	detector.py	Localizes 840x1820 ROI. Background luminance (<75) & Canny edge density (>0.028) filter camera cutaways.
3. Preprocessor	preprocessor.py	Grayscale conversion, CLAHE (clipLimit=2.5, tileGrid=(8,8)), and bilateral edge-preserving denoising.
4. OCR Engine	ocr_engine.py	PaddleOCR 3.7.0 (PP-OCRv6) deep learning text detection and alphanumeric character recognition.

5. Spatial Parser	parser.py	Centroid mapping into 4 horizontal rows and 11 columns. Proportional character splitting for merged boxes.
6. Temporal Aggregator	temporal_aggregator.py	Multi-frame sliding window voting per physical row index, monotonic scoring, and dynamic update tracking.
7. Consistency Validator	validator.py	Automated mathematical bowling rule verification across open frames, spare bonus, and strike carry.

3. Spatial Grid Mapping & Autonomous Name Discovery

Autonomous Name Discovery: When a bowler takes their turn, their full name appears in the top header banner ($\$Y \in [10, 100]$, $X \in [150, 600] \$$), while their corresponding row marker is highlighted in bright yellow/gold ($\$(R+G)/2 - B > 50 \$$). Temporal correlation discovers: **Row 1** → **JAGDISH**, **Row 2** → **VISHAL**, **Row 4** → **TARUN**. Row 3 had no active turn during this clip and is honestly preserved as **UNKNOWN_ROW_3**.

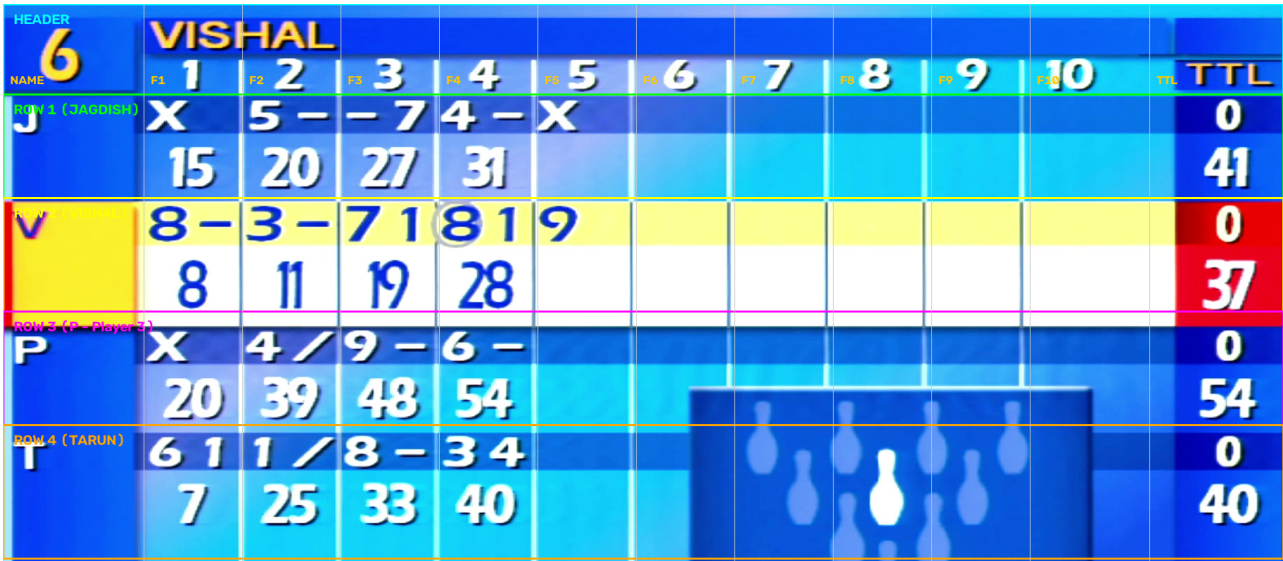


Figure 2: Calibrated 4-player spatial grid overlay partitioning rows, columns, and sub-cells.

4. Final Derived Scoreboard Results

The verified structured scoreboard generated by the production pipeline across all 4 players:

Player Name	Row	F1	F2	F3	F4	F5	F6-F10	TTL
JAGDISH	Row 1	X → 15	5- → 20	-7 → 27	4- → 31	X → 41	unplayed	41
VISHAL	Row 2	8- → 8	3- → 11	71 → 19	81 → 28	9- → 37	unplayed	37
UNKNOWN_ROW_3	Row 3	X → 20	4/ → 39	9- → 48	6- → 54	unplayed	unplayed	54
TARUN	Row 4	61 → 7	1/ → 25	8- → 33	34 → 40	unplayed	unplayed	40

5. Automated Mathematical Verification & Validation

The automated mathematical consistency checker (scoreboard_cv/validator.py) validates roll symbols against cumulative score deltas across all 4 players and played frames:

- Open Frames:** $\text{\$}\text{sum(rolls)} = \text{\$Cum}_f - \text{\$Cum}_{f-1}$ verified across all open frames.
- Spare Frames:** $\text{\$}10 + \text{\$Next Ball Pins} = \text{\$Cum}_f - \text{\$Cum}_{f-1}$ verified (e.g. Tarun F2 spare $10 + 8 = 18$; Row 3 F2 spare $10 + 9 = 19$).

- **Strike Frames:** $10 + \text{Next Two Balls Pins} = \text{Cum}_f - \text{Cum}_{f-1}$ verified (e.g. Jagdish F1 strike $10 + 5 = 15$; Row 3 F1 strike $10 + 10 = 20$).
- **Result: PASS — Zero mismatches** found across all played frames.

Deliverables: Codebase (GitHub: 8vimlesh/FOG-Assessment) | Demo Video (Google Drive / Git LFS) | JSON & CSV (output) | Documentation (PDF/MD).